



Interpret jazyka SOL26

Dokumentace k projektu IPP

Varianta – PHP, Python

Radim Pokorný (xpokorr00)

27. března 2026

Obsah

1	Úvod	2
2	Návrh	2
3	Datové struktury	4
4	Využití návrhových vzorů a principů OOP	4
4.1	Command a Message Dispatching	4
4.2	Řetězec zodpovědnosti	4
4.3	Singleton a Immutable Objects	5
5	Problémy a výzvy během implementace	5
5.1	Problém s logickými hodnotami	5
5.2	Kolize názvů metod a atributů	5
5.3	Rekurzivní hloubka a uzávěry	6
6	Zamyšlení nad možnostmi dalšího rozšíření	6
6.1	Podpora desetinných čísel	6
6.2	Iterační konstrukce Do a Foreach	6
7	Využití generativní umělé inteligence	6

1 Úvod

Tato dokumentace popisuje stručné specifikace implementace interpretu pro jazyk SOL26, pro který již byl vytvořen parser ve formátu XML. Cílem tohoto projektu je dodržet předem dané specifikace OO návrhu.

2 Návrh

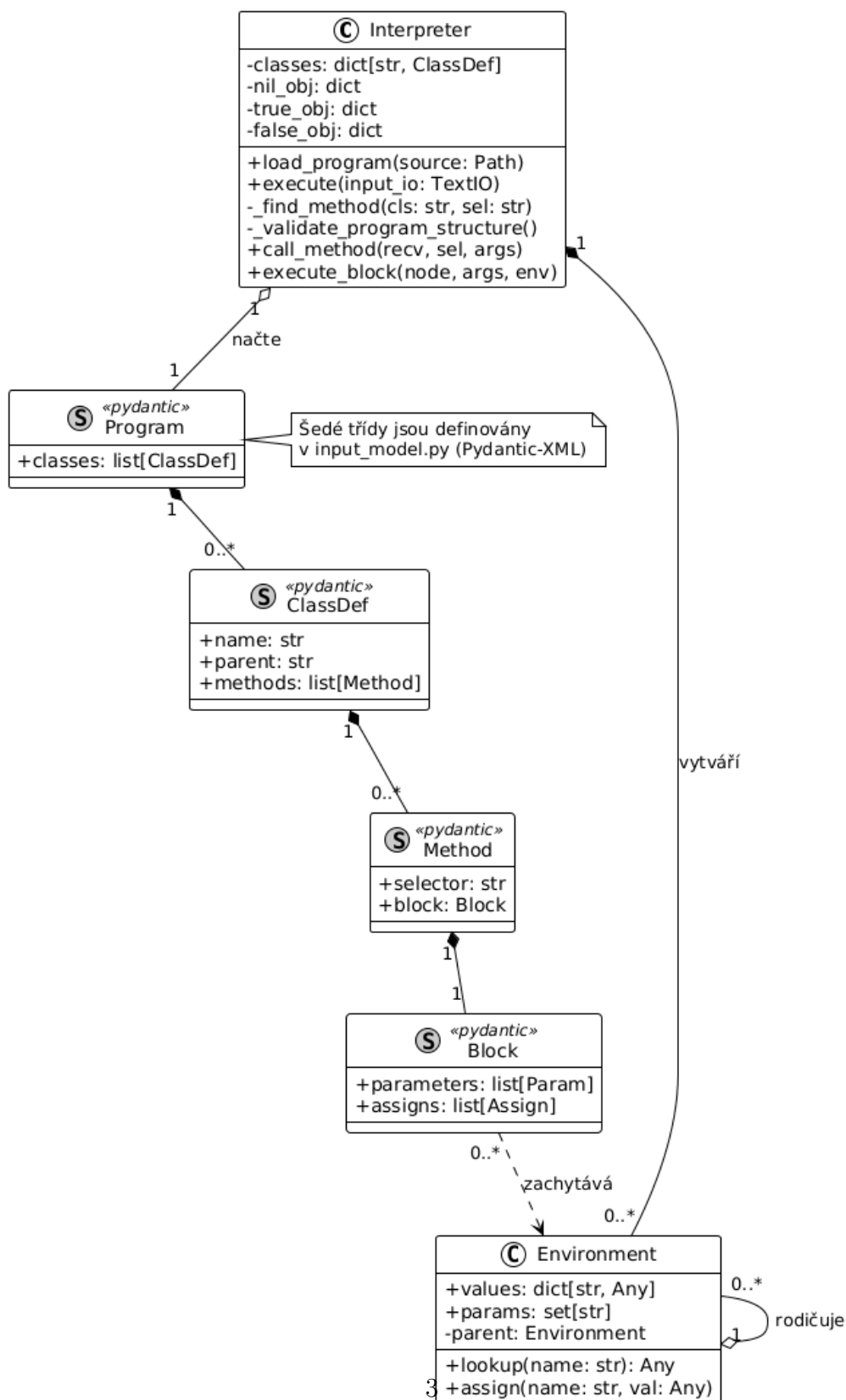
Systém interpretu pracuje ve třech fázích. Nejprve validuje vstup, poté provede sémantickou analýzu a nakonec vykoná program. Nejprve se rozebere XML strom, který je validován pomocí knihovny `pydantic-xml`. První kontrola je tedy provedena. Dále se provádí sémantická analýza, která ověřuje, zda je přítomna třída `Main` a hlavní metoda `run`. Kontroluje se také arita metod, kde je nutné splňovat pravidlo počtu dvojteček v selektoru s počty parametrů. Jakmile je provedena sémantická analýza, je buďto vrácen určený chybový stav nebo se přejde na rekurzivní vykonání programu, který využívá datovou strukturu `Environment`.

Celková architektura je řešena modulárně, kde se syntaktická validace, sémantická analýza a běh programu provádí na různých místech. Během návrhu byl kladen důraz i na přehlednost a využití tříd pro jednotlivé datové struktury, které jsou zmíněny v příští kapitole.

Po dokončení sestavení syntaktického stromu následuje fáze sémantické analýzy, kde interpret prochází deklarace všech objektů a metod. Tyto objekty jsou průběžně ukládány do slovníků, avšak prvním pravidlem je vždy ověření existence objektu `Main` a jeho metody `run`. Dále se provádí kontrola arity metod, přičemž v jazyce SOL26 je přítomnost znaku `:` poměrně velká obtíž, která si žádá důslednou režii.

Základem vykonávání programu je prochází syntaktického stromu rekurzivně. Během průchodu dochází k zasílání zpráv mezi objekty, kde se vykonávají instrukce. U nich musí být vždy jasné kdo je odesílá a její příjemce. Dále hledá implementaci vykonávané metody pomocí metody `lookup`, kde se směrem k rodiči postupně snaží dostat ke svému cíli, což umožňuje použití dědičnosti a polymorfismu.

Aby byl zachován správně rozsah platnosti, je využita implementovaná třída `Environment`. Každé volání metody a deklarace bloku vytváří novou instanci této třídy. Tato třída zajišťuje obecný přehled a je jedním z nejdůležitějších stavebních kamenů implementace.



Obrázek 1: UML diagram tříd interpretu SOL26

3 Datové struktury

Environment je datová struktura, která uchovává paměťový rámeček na rozsah platnosti proměnných. Obsahuje slovník hodnot a odkazuje na nadřazené prostředí téhož typu, tedy na svého rodiče. Díky této funkcionalitě umožňuje lexikální vyhledávání a uzávěry.

Interpreter je sice již převzatá datová struktura, ale byla doplněna o kompletní logiku životního cyklu programu. Používá metody pro vyhodnocování výrazů `evaluate_expr` a správu lokálních rozsahů v `execute_block`, kde využívá třídu **Environment**.

Veškeré objekty jsou v implementaci reprezentovány v datovém typu slovník, kde jsou uloženy názvy tříd, atributy i hodnoty.

4 Využití návrhových vzorů a principů OOP

Architektura projektu využívá OO paradigma pro lepší dekompozici složitějších problémů a jednodušší spolupráci entit. Místo procedurálního zpracování instrukcí je kladen důraz na zapouzdření do logických celků a definice rozhraní komponent.

4.1 Command a Message Dispatching

Tento vzor je v implementaci naprosto klíčový a jeho jádro je umístěno v metodě `call_method`. Interpret při každém volání zasílá zprávu svému příjemci, tedy objektu, kde pak na základě selektoru dohledá logiku v odpovídající třídě nebo tabulkách. Díky tomuto přístupu lze interpret v budoucnu rozšířit bez komplikací a nutnosti zasahovat přímo do implementace logiky.

4.2 Řetězec zodpovědnosti

Tento princip je schopen fungovat díky implementaci datové struktury **Environment**. Jakékoliv prostředí drží odkaz na svého rodiče a díky tomuto vzniká koncept lineární hierarchie, kde se požadavky typu `assign` nebo třeba `lookup` zpropagují nahoru, dokud se v příslušném prostředí nenalezne hledaný identifikátor. Tento princip je pro implementaci a optimalizaci velmi vhodný vzhledem k tomu, že by šlo ještě použít globální tabulku symbolů, což by bylo mnohem méně vhodné.

4.3 Singleton a Immutable Objects

Ve třídě `Interpreter` jsou vytvořeny instance objektů `nil`, `true` a `false` pouze jednou a jsou uloženy do atributů `nil_obj`, `true_obj` a `false_obj`. Všechny komponenty pak odkazují na tyto instance, což v jazyce Python hodně urychluje porovnání operátorem `is`. Díky tomuto vzoru se interpret chová konzistentně a zaručuje správnost logických operací kdekoliv v programu.

5 Problémy a výzvy během implementace

Během implementace interpretu bylo třeba pochopit více do hloubky jazyk Python a také přesně definovat typy proměnných v kódu, což přinášelo mnoho problémů. Dále některé koncepty OOP, na které byl brán větší ohled až v nerané fázi programování, způsobily mnoho komplexních výzev.

5.1 Problém s logickými hodnotami

Během testování interpretu jednotlivými programy došlo na vyhodnocení výrazu `false == false`, kde se jako výsledek očekává rovnost, ale výsledkem po několika pokusech byl stále `false`. Příčinou bylo, že interpret při každém výskytu literálu vytvořil nový slovník reprezentující objekt. Zadání vyžaduje, aby se porovnávala identita, nikoliv vnitřní hodnoty. Problém byl vyřešen zavedením statických objektů `self.true_obj`, `self.false_obj` a `self.nil_obj` v inicializaci interpretu. Poté, když se provádí jakékoliv porovnání, tak se porovnávají tyto jedinečné instance objektů.

5.2 Kolize názvů metod a atributů

Při definici stejným názvem např. getteru a metody si program nedokázal poradit s jejich odlišností a vrátil automaticky hodnotu atributu u getteru. Docházelo tedy k nepředvídatelnému chování v rámci dědičnosti. Řešením bylo přidání kontroly do metody `_handle_attribute_access`. Pokud se interpret pokusí přistoupit k atributu, nejprve pomocí `_find_method` ověří, zda v hierarchii tříd neexistuje metoda se stejným selektorem. Jestliže k tomu dojde, vrátí se sémantická chyba.

5.3 Rekurzivní hloubka a uzávěry

Při vykonání cyklů typu `whileTrue`: docházelo k chybnému hledání proměnných, kde byl problém u bloků. Bloky v cyklu ztrácely kontakt s třídou `Environment`. Blok v implementaci nesmí být vykonáván v prostředí svého volajícího, ale ve svém nadřazeném. Řešení bylo nakonec triviální, ale těžké k nalezení. Stačilo danému bloku přidat referenci na aktuální `Environment`.

6 Zamyšlení nad možnostmi dalšího rozšíření

Interpret myslí na nejrůznější případy a dalo by se tedy usoudit, že je poměrně komplexní a kompletní. Avšak jazyk SOL26 by mohl přinést jisté aktualizace, na které by byl interpret předem připraven.

6.1 Podpora desetinných čísel

Toto rozšíření by bylo poměrně jednoduché, ale i tak stále by bylo třeba dbát ohled na desetinnou tečku, která by mohla být špatně interpretována. Ideou by bylo přidat novou metodu `_create_obj` a s ní společně metodu `_handle_float_methods`. Jako poslední by se upravily podmínky pro typy operací, kde by se přidaly případy pro přetypování mezi typy `Float` a `Int`.

6.2 Iterační konstrukce Do a Foreach

Interpret pracuje s jednotlivými bloky bez chyby, což dává pevný základ všem možným iterátorům. Metoda `Do`: by postupně přidávala prvky sekvence čísel do bloku, což by přidalo pouze jednu metodu do kolekce `Dispatchers`. Třída `Environment` již uchovává vazbu na okolní proměnné, a tak by uživatel mohl elegantně měnit stav objektů mimo samotný blok.

7 Využití generativní umělé inteligence

Během implementace, sprovoznění projektu, návrhu Dockerfile apod. byly použity jazykové modely `ChatGPT` a `Gemini`. Pomohli mi také objasnit mé zkušenosti s OOP ze střední školy, které jsem si musel znovu připomenout a trochu obohatit. Dále mi byla velmi nápomocna při vysvětlení návrhových vzorů a pokročilejších úvah, kdy jsem potom měl lepší představu o implementaci. Konverzace s umělou inteligencí jsou přiloženy odkazem v souboru `ai-chatgpt.md` a `ai-gemini.md`.